

Lab 7: Policy-Based Reinforcement Learning

張苙烜/ 資科工碩 313551079

1 Introduction

這次 lab 要實作 policy-based RL, 不同於 value-based 可以更好的應對連續狀態空間。本次作業的 task1 A2C 和 task2/3 的 PPO 皆為 on policy 方法, 其模型更新僅依賴當下搜集的資料, 行為策略和目標策略相同, 可以提供一致的策略學習。作業共分為三個任務: Task 1 要求實作 A2C 並應用於經典控制任務 Pendulum-v1; Task 2 則進一步實作 PPO-Clip, 並結合 Generalized Advantage Estimation (GAE) 技術以改善訓練穩定性與樣本效率, 並同樣在 Pendulum-v1 上進行比較分析; Task 3 則挑戰將 PPO 延伸至較困難的 MuJoCo 任務 Walker2d-v4, 探討其在高維空間下的表現與超參數調整策略。

2 Implementation

2.1 Task1: Advantage Actor Critic (A2C) in a toy environment

A2C 是基於 Policy Gradient 和 Value Function 的學習方法, 分成 Actor 和 Critic network, 由 Actor 輸出動作 a 的機率分佈, Critic 輸出狀態 s 的 value, 再透過 Advantage 衡量策略的好壞 ($Advantage(s_t, a_t) = Q(s_t, a_t) - V(s_t)$)。但因為無法得知環境的細節, 所以此處的 $Q(s_t, a_t) = r_t + \gamma V(s_{t+1})$, 觀察目前執行該動作後得到的 reward r_t , 和 critic 估計出來下一個狀態的 value $V(s_{t+1})$ 近似未來的期望報酬。如果這個動作帶來的效益大於平均 value, 強化這個動作的選擇。

網路架構如下,

- Actor: 兩層架構 (3->64->64)
- Critic: 三層架構 (3->64->64->1)

在 Actor network 中要實作 stochastic policy, 從 normal distribution 中隨機抽取動作 (`dist.sample()`) 讓 agent 可以嘗試更多不同的策略增加 exploration。

```
1 class Actor(nn.Module):
2     def __init__(self, in_dim: int, out_dim: int):
3         """Initialize."""
4         super(Actor, self).__init__()
5
6         #####TODO#####
7         # Remember to initialize the layer weights
8         self.fc1 = nn.Linear(in_dim, 64)
9         self.fc2 = nn.Linear(64, 64)
10        self.mu = nn.Linear(64, out_dim)
11        self.std = nn.Linear(64, out_dim)
12
13        initialize_uniformly(self.mu)
14        initialize_uniformly(self.std)
15        #####
16
17    def forward(self, state: torch.Tensor) -> torch.Tensor:
18        """Forward method implementation."""
19
20        #####TODO#####
21        x = F.relu(self.fc1(state))
```

```

22     x = F.relu(self.fc2(x))
23
24     mu = torch.tanh(self.mu(x)) * 2.0 # pendulum的動作輸出再 [-2,2] 之間
25     std = F.softplus(self.std(x)) + 1e-5
26
27     dist = Normal(mu, std)
28     action = dist.sample() # Sample an action from the distribution
29     #####
30
31     return action, dist
32
33
34 class Critic(nn.Module):
35     def __init__(self, in_dim: int):
36         """Initialize."""
37         super(Critic, self).__init__()
38
39         #####TODO#####
40         # Remeber to initialize the layer weights
41         self.model = nn.Sequential(
42             nn.Linear(in_dim, 64),
43             nn.ReLU(),
44             nn.Linear(64, 64),
45             nn.ReLU(),
46             nn.Linear(64, 1),
47         )
48
49         #####
50
51     def forward(self, state: torch.Tensor) -> torch.Tensor:
52         """Forward method implementation."""
53         #####TODO#####
54         value = self.model(state)
55         #####
56         return value

```

在 update model 中，每個 episode agent 根據當前 policy 執行動作並與環境互動，將 (state, action, reward, next_state, done) 暫存於 self.transition，再根據當下經驗即時更新 model (on-policy setting) · 計算 TD error 做為 critic network 的學習目標，計算 MSEloss 更新 critic · Actor 透過 policy gradient 搭配 log-probability 計算 loss，其公式如下：

$$\mathcal{L}_{critic} = (r_t + \gamma V(s_{t+1}) - V(s_t))^2 \quad (1)$$

$$\mathcal{L}_{actor} = -\log\pi(a_t|s_t) \cdot TDerror \quad (2)$$

在計算 policy_loss 時有加上 entropy，讓 policy 在初始階段有隨機性。

$$\mathcal{L}_{actor} = -\log\pi(a_t|s_t) \cdot TDerror - \beta \cdot Entropy \quad (3)$$

```

1     def update_model(self) -> Tuple[torch.Tensor, torch.Tensor]:
2         """Update the model by gradient descent."""
3         state, log_prob, next_state, reward, done = self.transition
4
5         # Q_t    = r + gamma * V(s_{t+1}) if state != Terminal
6         #        = r                      otherwise
7         mask = 1 - done
8
9         #####TODO#####
10        # value_loss = ?
11        state = state.unsqueeze(0)

```

```

12     next_state = torch.FloatTensor(next_state).unsqueeze(0).to(self.device)
13     reward = torch.FloatTensor([reward]).to(self.device)
14     done = torch.FloatTensor([done]).to(self.device)
15
16     value = self.critic(state)
17     # next_value = self.critic(next_state)
18     with torch.no_grad():
19         next_value = self.critic(next_state)
20         target = reward + mask * self.gamma * next_value
21
22
23     # TD target : r + gamma * V(s_{t+1}) * mask
24     # target = reward + self.gamma * next_value * mask
25     td_error = target - value
26
27     # critic loss
28     value_loss = td_error.pow(2).mean()
29
30     #####
31
32     # update value
33     self.critic_optimizer.zero_grad()
34     value_loss.backward()
35     torch.nn.utils.clip_grad_norm_(self.critic.parameters(), max_norm=1.0)
36     self.critic_optimizer.step()
37
38     # advantage = Q_t - V(s_t)
39     #####TODO#####
40     # policy_loss = ?
41     entropy = -self.transition[1].exp() * self.transition[1] # log_prob * prob =
42     -entropy
43     policy_loss = -(log_prob * td_error.detach()) - self.entropy_weight * entropy.
44     mean()
45     #####
46     # update policy
47     self.actor_optimizer.zero_grad()
48     policy_loss.backward()
49     torch.nn.utils.clip_grad_norm_(self.actor.parameters(), max_norm=1.0)
50     self.actor_optimizer.step()
51
52     return policy_loss.item(), value_loss.item()

```

2.2 Task2 & 3:PPO-Clip

Proximal Policy Optimization (PPO) 是一種強健的 policy-based 強化學習方法，其設計目標是穩定且有效率地更新 policy。在本次實作中，我們採用的是 PPO-Clip 版本，搭配 Generalized Advantage Estimator (GAE) 來減少 variance 並提升學習效率。與 A2C 相比，PPO 並非每個 step 都立即更新，而是先進行 rollout 蒐集一定長度的資料，再進行多次 mini-batch 更新，能夠有效重複利用資料並提升樣本效率。本次 lab 為進一步驗證 PPO 在高維控制任務上的表現，我們將環境從 Pendulum-v1(task2) 擴展至 Walker2d-v4(task3)。

2.2.1 Sampling

透過與環境互動來收集一整段 trajectory (軌跡) 作為訓練樣本。在 train() 函數中，每次迴圈透過 select_action(state) 抽樣動作，並在其中儲存目前狀態 state、抽樣出的動作 action、由 Critic 預測的 state value $V(s_t)$ ，以及動作的 log probability $\log(a_t|s_t)$ ，分別儲存至 self.states、self.actions、self.values、self.log_probs 的 list 中，這些資料會在 rollout 結束時一次由 update model 進行訓練更新，透過 mini-batch

多次重複利用資料來提升樣本效率。

2.2.2 GAE : Generalized Advantage Estimation

GAE 計算每個時間步的優勢值 (advantage)，以提升訓練的穩定性與效率。GAE 透過結合多步的 TD error，並加入一個控制 bias-variance 折衷的超參數 τ ，計算出更平滑的 advantage 值。從最後一個 episode 逆推，累積每一步的 TD error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ ，再以 γ 與 τ 的加權折扣進行平滑，公式 $A_t = \delta_t + \gamma\tau\delta_{t+1} + (\gamma\tau)^2\delta_{t+2}\dots$ ，最後再加上 $V(s_t)$ 回傳 returns。

```

1 def compute_gae(
2     next_value: list, rewards: list, masks: list, values: list, gamma: float, tau:
3     float) -> List:
4     """Compute gae."""
5     values = values + [next_value]
6     gae = 0
7     returns = []
8
9     for step in reversed(range(len(rewards))):
10         delta = rewards[step] + gamma * values[step + 1] * masks[step] - values[step]
11         gae = delta + gamma * tau * masks[step] * gae
12         returns.insert(0, gae + values[step])
13
14     return returns

```

2.2.3 clipped objective

為了防止策略更新變動太大，提高訓練穩定性，使用一個 ratio 比對新舊 policy difference

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} = \exp(\log\pi_\theta - \log\pi_{\theta_{old}}) \quad (4)$$

loss function :

$$\mathcal{L}_{clip} = E_t[\min(ratio \cdot TDerror, clip(ratio, 1 - \epsilon, 1 + \epsilon) \cdot TDerror)] \quad (5)$$

這邊一樣有加上 entropy，critic 更新與 a2c 相同。

```

1     # calculate ratios
2     _, dist = self.actor(state)
3     log_prob = dist.log_prob(action).sum(dim=-1, keepdim=True)
4     ratio = (log_prob - old_log_prob).exp()
5
6     # actor_loss
7     #####TODO#####
8     # actor_loss = ?
9     loss = ratio * adv
10    clipped_loss = torch.clamp(ratio, 1. - self.epsilon, 1. + self.epsilon) * adv
11    actor_loss = -torch.min(loss, clipped_loss).mean() - self.entropy_weight * dist.
12    entropy().mean()
13    #####
14
15    # critic_loss
16    #####TODO#####
17    # critic_loss = ?
18    cur_value = self.critic(state)
19    critic_loss = F.mse_loss(return_, cur_value)
20    #####
21
22    # train critic
23    self.critic_optimizer.zero_grad()

```

```

23     critic_loss.backward(retain_graph=True)
24     # clip gradient
25 torch.nn.utils.clip_grad_norm_(self.critic.parameters(), 0.5)
26     self.critic_optimizer.step()
27     # train actor
28     self.actor_optimizer.zero_grad()
29     actor_loss.backward()
30     # clip gradient
31 torch.nn.utils.clip_grad_norm_(self.actor.parameters(), 0.5)
32     self.actor_optimizer.step()

```

3 Analysis and discussions

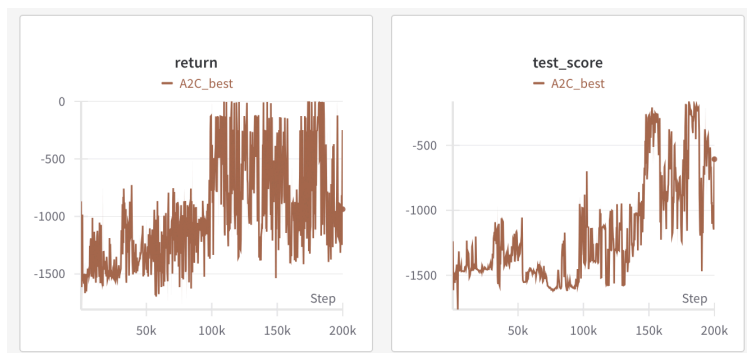
3.1 Training Curves

3.1.1 Task1

參數設定：

- Actor Learning Rate: 4e-4
- Critic Learning Rate: 4e-3
- Discount Factor (γ): 0.99
- Entropy Coefficient (β): 1e-2
- Entropy Decay: 0.95

從下圖可以看到在 100k 左右 training 的 return 就急速上升，但是後面還是會震盪。右圖的 test_score 是每部都加入 20 次 random seed test，結果顯示隨者 step 增加，test score 不段上升，顯示 A2C 有正常收斂。



((a)) A2C Training Curves on Pendulum

```

-----
Episode 0: Total Reward = -131.31038334952726
Episode 1: Total Reward = -123.40343738509205
Episode 2: Total Reward = -116.5954511953883
Episode 3: Total Reward = -229.5776993289999
Episode 4: Total Reward = -2.2505396657671994
Episode 5: Total Reward = -139.39391159922678
Episode 6: Total Reward = -119.22390135579538
Episode 7: Total Reward = -125.46686632116288
Episode 8: Total Reward = -132.8919857626593
Episode 9: Total Reward = -2.303748324757374
Episode 10: Total Reward = -127.33776862875162
Episode 11: Total Reward = -225.72256903632376
Episode 12: Total Reward = -255.71387292126
Episode 13: Total Reward = -231.56130534029953
Episode 14: Total Reward = -131.3759773769538
Episode 15: Total Reward = -131.09908243175573
Episode 16: Total Reward = -125.26162364141346
Episode 17: Total Reward = -121.51351718214833
Episode 18: Total Reward = -131.05608879590548
Episode 19: Total Reward = -122.68059683137025
Average score over 20 episodes: -136.29

```

((b)) Result Screenshot

Figure 1: Training curve and result screenshot for A2C on Pendulum

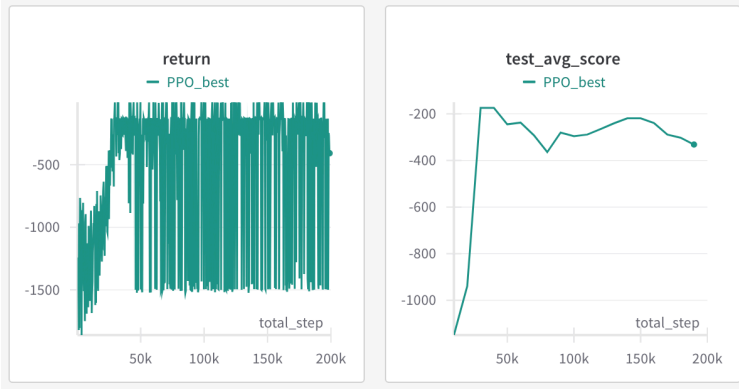
3.1.2 Task2

Task2 改用 ppo 跑 pendulum，參數設定如下

- Actor Learning Rate: 5e-4
- Critic Learning Rate: 5e-3
- Discount Factor (γ): 0.9

- epsilon: 0.2
- tau: 0.8

比較 Task1 和 task2，可以發現 PPO 提升了將近一半的 sample efficiency，同一個任務花了一半得 step 就收斂，這是因為 Clipped Objective 限制新舊 policy 變動幅度，避免出現過大的 policy shift，讓每次更新更穩定有效。Advantage Normalization 對 Advantage 做標準化，減少 scale 對訓練的影響，使模型學習更穩定且收斂速度更快。GAE (Generalized Advantage Estimation) 引入 λ -return 提高 advantage 的估計品質，在 bias 與 variance 之間取得平衡，進一步提升訓練效果。



((a)) PPO Training Curves on Pendulum

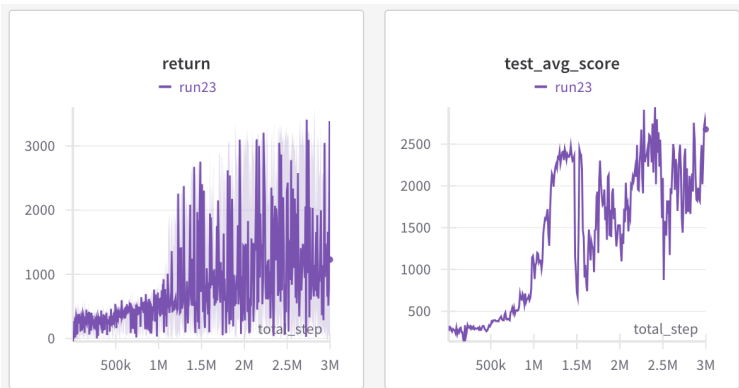
```
c = pygame.math.Vector2(1,0)
Test 1/20, Score: -256.67
/home/drapeegoo/anaconda3/envs/
appers/rendering.py:283: UserW
edia/drapeegoo/E/DLP/Lab7/ppo_
o_folder' for the 'RecordVideo
logger.warn(
Test 2/20, Score: -130.06
Test 3/20, Score: -126.08
Test 4/20, Score: -131.49
Test 5/20, Score: -258.66
Test 6/20, Score: -136.11
Test 7/20, Score: -130.89
Test 8/20, Score: -130.78
Test 9/20, Score: -2.47
Test 10/20, Score: -126.85
Test 11/20, Score: -127.15
Test 12/20, Score: -130.44
Test 13/20, Score: -261.64
Test 14/20, Score: -132.63
Test 15/20, Score: -277.49
Test 16/20, Score: -129.72
Test 17/20, Score: -135.72
Test 18/20, Score: -127.85
Test 19/20, Score: -127.88
Test 20/20, Score: -2.65
Average Score: -144.16
```

((b)) Result Screenshot

Figure 2: Training curve and result screenshot for PPO on Pendulum

3.1.3 Task3

Walker 相較 Pendulum 更高維度，要學習的是行走行為，不同於 Pendulum 的角度擺臂控制，是一個連續狀態動作的平衡，因此收斂較困難，sample efficiency 更重要。從 ?? 可以觀察到 1M 之後 return 會急速上升開始收斂，而 20 次的 test average 也可以在 1M 到 1.5M 之間突破 2500，test score 穩健上升，顯示出良好的學習效果。



((a)) PPO Training Curves on Walker

```
Test 1/20, Score: 2532.44
/home/drapeegoo/anaconda3/envs/la
vs/registration.py:519: Deprecati
is out of date. You should consi
logger.deprecation(
Test 2/20, Score: 2509.48
Test 3/20, Score: 2539.62
Test 4/20, Score: 2478.42
Test 5/20, Score: 2451.77
Test 6/20, Score: 2547.71
Test 7/20, Score: 2592.03
Test 8/20, Score: 2522.90
Test 9/20, Score: 2650.19
Test 10/20, Score: 2499.43
Test 11/20, Score: 2606.76
Test 12/20, Score: 2523.32
Test 13/20, Score: 2590.86
Test 14/20, Score: 2427.65
Test 15/20, Score: 2431.65
Test 16/20, Score: 2411.50
Test 17/20, Score: 2484.46
Test 18/20, Score: 2428.34
Test 19/20, Score: 2674.63
Test 20/20, Score: 2471.85
Average Score: 2518.75
```

((b)) Result Screenshot

Figure 3: Training curve and result screenshot for PPO on Walker

3.2 Empirical Analysis of Clipping Parameter and Entropy Coefficient

為了瞭解 PPO 演算法中關鍵超參數對學習表現的影響，我們針對 clipping 係數 ϵ (`-epsilon`) 與 entropy 權重 β (`-entropy-weight`) 進行一系列實驗，觀察其對於學習穩定性與最終 reward 的影響，藉此找出能有效提升樣本效率與策略穩健性的參數組合。我嘗試了不同的組合，分別是 `entropy weight = 0.001, 0.01, 0.005`。entropy weight 決定了 policy 的隨機性，當他太大太隨機無法穩定學會好策略，太小則會過度快速收斂特定動作 return 不再上升。epsilon 決定了策略更新的範圍，太小政策更新非常保守，穩定但收斂慢；太大則是更新幅度大，學習快速但容易不穩定，這邊選用 0.2 為最優解。

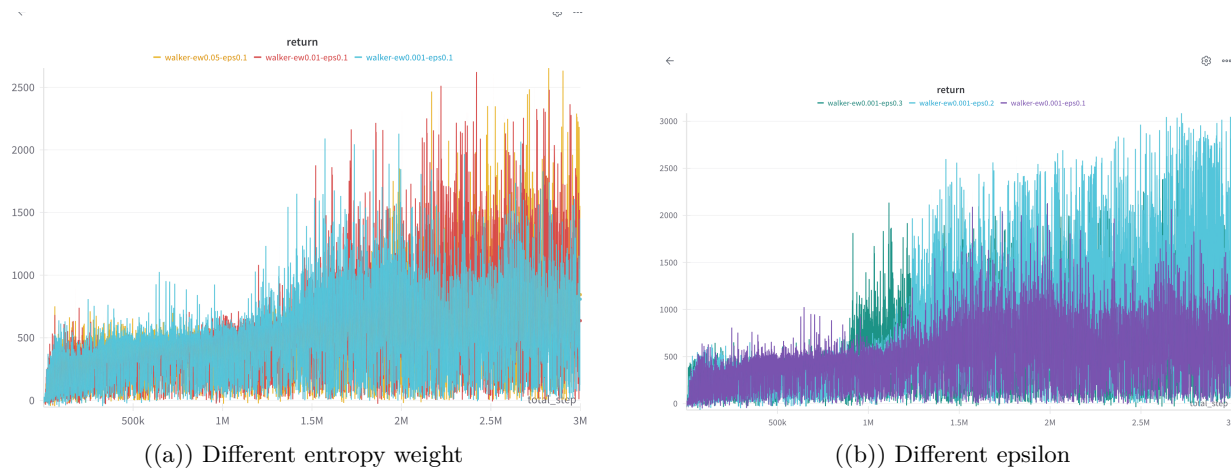


Figure 4: Comparison of performance under different entropy weight and epsilon

3.3 Additional analysis on other training strategies

在練習過程中發現，強化學習學習中需要設定 random 環境讓他去適應不同初始狀態的學習過程，這樣他才能更好的應對不同初始狀態的測試，否則會過度擬合某一類起始狀態的訓練策略結果，並沒有真的學會策略。但 random 起始狀態每次訓練出來同個參數可能也會有不同的結果，甚至不同裝置就會有相差迥異的結果，這點是比較困擾的因素。推測原因是因為強化學習跟環境很有關係，多變化的環境可以讓他學習到比較好的結果，所以我覺得可以嘗試 seed 的分佈多樣化，不要完全 random，以更好地控制訓練結果。